Advanced Materials of Python

yen yung chin

March 10, 2020

Contents

1 Python 套件管理 **3**
1.1 pip v.s. conda . . . 3
1.1.1 pip . . . 3
1.1.2 conda . . . 3
1.1.3 In both cases: . . . 3
4subsection.1.2
4subsubsection.1.2.1
4subsubsection.1.2.2
1.3 conda . . . 5
1.3.1 下載 v.s. 安裝 . . . 5
1.3.2 常見 conda 指令 . . . 5
1.4 python package 安裝 . . . 6
1.4.1 安裝 package: . . . 6
1.4.2 移除 package: . . . 6
1.4.3 安裝特定版本 python . . . 6
1.4.4 了解目前系統可用套件 . . . 6
6subsection.1.5
1.5.1 建立虛擬環境 . . . 6
1.5.2 啟用某個虛擬環境 . . . 6
1.5.3 退出某個虛擬環境 . . . 6
1.5.4 刪除某個虛擬環境 . . . 6
1.5.5 列出目前系統中所有的虛擬環境 . . . 6

2 Numpy 模組 **6**
2.1 Numpy . . . 6
2.2 匯入模組 . . . 7
2.3 Create ndarray . . . 7
2.3.1 一維陣列 . . . 7
2.3.2 多維陣列 . . . 7
2.4 矩陣運算 . . . 8
2.5 特殊矩陣 . . . 8
2.6 ndarray 屬性 . . . 9
2.6.1 練習 . . . 9
2.7 矩陣形狀轉換 . . . 10
2.8 矩陣疊加 . . . 10
2.9 ndarray 轉 list . . . 10
2.10 Numpy 常用統計函數 . . . 11

3 Matplotlib **11**
3.1 Matplotlib 簡介 11
3.2 折線圖 11
3.2.1 基本語法 11
3.2.2 練習一：繪製折線圖 12
3.2.3 圖形美化: plot() function 12
3.2.4 練習 2：圖表美化 15
3.3 bar chart: plt.bar() 15
3.3.1 語法 15
3.3.2 練習 3： 15
3.4 pie chart: plt.pie() 15
3.4.1 語法： 15
3.4.2 參數： 15
3.4.3 範例 16
3.5 其它圖表 16
3.6 文字註解: plt.text() 16
3.6.1 語法 16
3.6.2 範例 17
3.7 子圖表: plt.subplot() 17

4 區域資料檔分析 **19**
4.1 Excel/CSV: Pandas 模組 19
4.1.1 CSV 19
4.1.2 Pandas 19
4.1.3 Pandas 資料結構 19
4.2 Pandas functions 20
20subsubsection.4.2.1
4.2.2 條件式選取資料 21
4.3 進階分析 21
4.3.1 計算: Aggregation, Groupby 21
4.3.2 資料檔下載: `scores.csv` 22
4.4 資料視覺化 22
4.4.1 Bar chart 22
4.4.2 Pie chart 22
4.4.3 Scatter chart 23
4.4.4 Line chart 23

5 網路爬蟲 **25**
5.1 parse JSON 25
25subsubsection.5.1.1
5.1.2 JSON 的優點 25
5.1.3 JSON 結構 26
5.1.4 實作 1: 國際主要國家貨幣每月匯率概況 26
5.1.5 實作 2: 澎湖生活博物館每月參觀人次統計資料 27
5.1.6 實作 3: 下載 JSON 28
5.1.7 實作: 解析 JSON 28
5.1.8 練習 29
5.2 parse HTML 29
5.2.1 所需套件 29
5.3 範例: 以程式抓取 PTT 電影板 header 30

5.3.1 原則 30
5.3.2 實作: 取得 HTML 資料 30
5.3.3 實作：解析 HTML 資料內容 30
5.4 參考資料 31

6 Python 學習資源 **31**

7 0802 CNN **31**

8 如何把 python 程式 publish 成網站應用程式 **31**

1 Python 套件管理

Python 的一大優勢之一便是龐大的第三方函式庫，讓使用 python 的程式設計師可以方便的呼叫、進行如網路資料下載解析、資料的視覺化、甚或是大數據的複雜分析與人工智慧的 balabala....，目前用來管理這些龐大套件的工具主要有二：pip 與 Conda。

1.1 pip v.s. conda

1.1.1 pip

- Python packages only.
- Compiles everything from source. EDIT: pip now installs binary wheels, if they are available.
- Blessed by the core Python community (i.e., Python 3.4+ includes code that automatically bootstraps pip).

1.1.2 conda

- Python agnostic. The main focus of existing packages are for Python, and indeed Conda itself is written in Python, but you can also have Conda packages for C libraries, or R packages, or really anything.
- Installs binaries. There is a tool called conda build that builds packages from source, but conda install itself installs things from already built Conda packages.
- External. Conda is the package manager of Anaconda, the Python distribution provided by Continuum Analytics, but it can be used outside of Anaconda too. You can use it with an existing Python installation by pip installing it (though this is not recommended unless you have a good reason to use an existing installation).

1.1.3 In both cases:

- Written in Python

- Open source (Conda is BSD and pip is MIT)

1.2 Conda, Anaconda, Minconda[1]

1.2.1 difference between conda, anaconda, and miniconda [2]

- conda is both a command line tool, and a python package.
- Miniconda installer = Python + conda
- Anaconda installer = Python + conda + meta package anaconda
- conda is an environment manager and a package manager. It means the tool itself. conda makes it possible to
  - install package with conda install flake8
  - create an environment with any version of Python with conda create -n myenv python=3.6
- conda is not a binary command, is a Python package. To make conda work, you have to create a Python environment and install package conda into it. This is where Anaconda installer and Miniconda installer comes in.
- Installer Minoconda installs a Python and the package conda. Installer Anaconda not only does what Miniconda does, it also install a meta Python package named anaconda for you.
- 不一定要安裝 Anaconda 或 Miniconda，也可透過 pip 直接安裝 conda

```
pip install conda
```

1.2.2 conda、miniconda、anaconda 的區別[3]

- conda 是一種通用包管理系統，旨在構建和管理任何語言和任何類型的軟件。舉個例子：包管理與 pip 的使用類似，環境管理則允許用戶方便地安裝不同版本的 python 並可以快速切換。
- Anaconda 則是一個打包的集合，裡面預裝好了 conda、某個版本的 python、眾多 packages、科學計算工具等等，就是把很多常用的不常用的庫都給你裝好了。
- Miniconda，顧名思義，它只包含最基本的內容——python 與 conda，以及相關的必須依賴項，對於空間要求嚴格的用戶，Miniconda 是一種選擇。就只包含最基本的東西，

[1] Anaconda、Miniconda、Conda、pip 的相互關係 (轉載)

[2] What are the differences between Conda and Anaconda?

[3] conda、miniconda、anaconda 的區別以及在 pycharm 中選擇 conda 的虛擬環境

其他的庫得自己裝。

1.3 conda

Conda 是一個開源的跨平台工具軟體，它被設計作為 Python、R、Lua、Scala 與 Java 等任何程式語言的套件、依賴性以及工作環境管理員，特別受到以 Python 作為主要程式語言的資料科學團隊所喜愛。傳統 Python 使用者以 pip 作為套件管理員（package manager）、以 venv 作為工作環境管理員（environment manager），而 conda 則達成了「兩個願望、一次滿足」既可以管理套件亦能夠管理工作環境。在這個小節我們介紹如何使用 conda 最重要的幾個核心功能，讓資料科學團隊重現指定的工作環境設定。[4]

1.3.1 下載 v.s. 安裝

- Anaconda
- Miniconda

1.3.2 常見 conda 指令

- conda –version 檢視 conda 版本
- conda update $\text{PACKAGE}_{\text{NAME 更新指定套件}}$
- conda –help 檢視 conda 指令說明文件
- conda list –ENVIRONMENT 檢視指定工作環境安裝的套件清單
- conda install $\text{PACAKGE}_{\text{NAME}}$=MAJOR.MINOR.PATCH 在目前的工作環境安裝指定套件
- conda remove $\text{PACKAGE}_{\text{NAME}}$ 在目前的工作環境移除指定套件
- conda create –name ENVIRONMENT python=MAIN.MINOR.PATCH 建立新的工作環境且安裝指定 Python 版本
- conda activate ENVIRONMENT 切換至指定工作環境
- conda deactivate 回到 base 工作環境
- conda env export –name ENVIRONMENT –file ENVIRONMENT.yml 將指定工作環境之設定匯出為.yml 檔藉此複製且重現工作環境

[4]輕鬆學習 Python：conda 的核心功能

1.4 python package 安裝

1.4.1 安裝 package:

```
conda install pandas
```

1.4.2 移除 package:

```
conda remove pandas
```

1.4.3 安裝特定版本 python

```
conda install python=3.5
```

1.4.4 了解目前系統可用套件

1.5 python 執行環境建立與維護[5]

1.5.1 建立虛擬環境

```
conda create -n myDLProject
```

1.5.2 啟用某個虛擬環境

```
source activate myDLProject
```

1.5.3 退出某個虛擬環境

```
source deactiveate
```

1.5.4 刪除某個虛擬環境

```
conda env remove -n myDLProject
```

1.5.5 列出目前系統中所有的虛擬環境

2 Numpy 模組

2.1 Numpy

Numpy 是 Python 的一個重要模組，主要用於資料處理上。Numpy 底層以 C 和 Fortran 語言實作，所以能快速操作多重維度的陣列。當 Python 處理龐大資料時，其原生 list 效能表現並不理想（但可以動態存異質資料），而 Numpy 具備平行處理的能力，可以將操作動作一次套用在大型陣列上。此外 Python 其餘重量級的資料科學相關套件（例如：Pandas、SciPy、Scikit-learn 等）都幾乎是奠基在 Numpy 的基礎上。因此學會 Numpy 對於往後學習其他資料科學相關套件打好堅實的基礎。[6]

[5] Day01 Anaconda 環境安裝!

[6] 從零開始學資料科學：Numpy 基礎入門

2.2 匯入模組

- 使用模組裡的函式要加模組名稱，如 numpy.arrry( )。例：
- 匯入 numpy 模組並使用 np 作為簡寫，這是 Numpy 官方倡導的寫法
- Numpy 中的多維資料型別稱為 ndarray

2.3 Create ndarray

Numpy 的重點在於陣列的操作，其所有功能特色都建築在同質且多重維度的 ndarray（N-dimensional array）上。ndarray 的關鍵屬性是維度（ndim）、形狀（shape）和數值類型（dtype）。一般我們稱一維陣列為 vector 而二維陣列為 matrix。一開始我們會引入 numpy 模組，透過傳入 list 到 numpy.array() 創建陣列。[6]

2.3.1 一維陣列

- list 或 tuple 轉入

```
import numpy as np
np1 = np.array( [1, 2, 3, 4] )
print(np1)
```

- 使用 np.arange( ) 方法

```
import numpy as np
np2 = np.arange(5)
print(np2)
```

2.3.2 多維陣列

- 二維陣列

```
import numpy as np
np4 = np.array( [[1, 2, 4], [3,4,5]])
print('np4:', np4)

np5 = np.array([np.arange(3), np.arange(3)])
print('np5:', np5)

np6 = np.arange(8).reshape(2, 4)
print('np6:', np6)
```

- 多維陣列

```
import numpy as np

np7 = np.arange(24).reshape(2, 3, 4)
print('np7:',np7)
```

- 隨機矩陣

```python
import numpy as np

np8 = np.random.random( (3, 2)) #矩陣大小以tuple表示
print('np8:', np8)

np9 = np.random.randint( 0, 100, size=[4, 3]) #矩陣大小以list表示
print('np9:', np9)
```

2.4 矩陣運算

```python
import numpy as np

A = np.array([
    [5, 3, 7],
    [0, 1, -6]
])

B = np.array([
    [1, 2, 3],
    [-1, -2, -3]
])

C = np.array([
    [1, 2],
    [-1, -2],
    [0, 5]
])

k = -3

print('---A+B---\n', A+B)
print('---A-B---\n', A-B)
print('---k*B---\n', k*B)
print('---A.dot(C)---\n', A.dot(C))
```

2.5 特殊矩陣

- 元素全為 0 矩陣:
- 元素全為 1 矩陣:
- 單位矩陣 (Identity):
- 反矩陣 (Inverse):

```python
import numpy as np

print('np.zeros:\n', np.zeros((3,5), dtype='int'))
print('np.ones:\n', np.ones((3,5), dtype='int'))
print('np.eyes:\n', np.eye(4, dtype='int'))
x = np.random.randint( 0, 10, size=[3, 3])
print('x:\n', x)
xt = x.T # 轉置
print('xt:\n', xt)
mat = xt.dot(x)
```

```
print('mat:\n', mat)
print('np.linalg.inv(mat):\n', np.linalg.inv(mat))
```

2.6 ndarray 屬性

- dtype：ndarray 每個元素的資料型別 (可於創建矩陣時同時指定)
  - int, float, bool, D(複數)
- ndim：ndarray 的維度
- shape：ndarray 的大小，即矩陣 m*n (m 列 n 行)
- size：ndarray 的元素個數
- itemsize：ndarray 中每個元素記憶體大小 (單位 byte)

```
import numpy as np

np1 = np.arange(12, dtype='D').reshape(3, 2, 2)
print(np1)
print('dtype=', np1.dtype)
print('shape=', np1.shape)
print('size=', np1.size)
print('itemsize=', np1.itemsize)
print('nbytes=', np1.nbytes)
```

- nbytes：ndarray 的記憶體大小 (單位 byte)
  - real 和 imag：複數的實部和虛部
  - real 回傳複數的實部
- imag 回傳複數的虛部
- T：矩陣轉置

2.6.1 練習

- 使用 random 函式產生 3*5 的矩陣 A
- 使用 randint 函式產生 5*3 的矩陣 B
- 使用 arange 和 reshape 函式產生 3*5 的矩陣 C
- 使用 list 自建 5*3 的矩陣 D
- 計算 10*A + C 和 B - D
- 計算 B • C 和 C • D

- 比較 (B • C)T 和 CT • BT 是否相同?
- 求矩陣 E 的反矩陣

2.7 矩陣形狀轉換

- reshape(m, n, ⋯)：不改變原矩陣形狀
- resize(m, n, ⋯)：和 reshape 類似，但會改變原矩陣形狀
- ravel( ) / flatten( )：將多維矩陣轉換成一維矩陣，原矩陣不變
- shape = (m, n, ⋯)：用 tuple 來指定矩陣大小，會改變原矩陣

```
import numpy as np
a = np.arange(12)
print(a)
print(a.reshape(3,4))
print(a)
print(a.resize(3,4))
print(a)
print(a.flatten())
print(a)
a.shape = (2, 6)
print(a)
```

2.8 矩陣疊加

- 水平疊加：np.hstack(<tuple, list>)
- 垂直疊加：np.vstack(<tuple, list>)
- 深度疊加：np.dstack(<tuple, list>) 以此例，a, b 原為 2*6 矩陣， 深度疊加後變成 2*6*2 矩陣。

```
import numpy as np
a = np.arange(12).reshape(2, 6)
b = a * 2

print(a)
print(b)

print(np.hstack([a, b]))
print(np.vstack([a, b]))
print(np.dstack([a, b]))
```

2.9 ndarray 轉 list

- ndarray 轉 list

```
import numpy as np
a = np.arange(12).reshape(4, 3)
print(a)
b = a.tolist()
print(b)
```

2.10 Numpy 常用統計函數

- np.sum(<np>)：回傳和
- np.mean(<np>)：回傳平均值
- np.max(<np>)：回傳最大值
- np.min(<np>)：回傳最小值
- np.ptp(<np>)：回傳最大值減最小值
- np.std(<np>)：回傳標準差
- np.var(<np>)：回傳變異數
- np.cumsum(<np>)：回傳累加值
- np.cumprod(<np>)：回傳累積值　註：使用時需要指定 axis 軸的方向，若不指定，預設統計整個 numpy

3 Matplotlib

3.1 Matplotlib 簡介

- NumPy 的可視化操作界面
- 為 Python 最多人使用的 2D 繪圖工具
- 優點：圖形美觀、類型多、相容於 Matlab
- 官方社群網站 `https://matplotlib.org/`
- 維基百科 `https://zh.wikipedia.org/wiki/Matplotlib`

3.2 折線圖

3.2.1 基本語法

- import matplotlib.pyplot as plt

- plt.plot( ) 函式為 matplotlib.pyplot 模組畫線條方法，其語法如下
- np.sin( ) 函式為 Numpy 模組求正弦值
- plt.show( ) 函式用來顯示圖形

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(-3, 3, 0.01)
plt.clf()
plt.plot(x, np.sin(x))
# for web-based
#plt.show()

plt.savefig('SimpleSin.png', dpi=300)
```

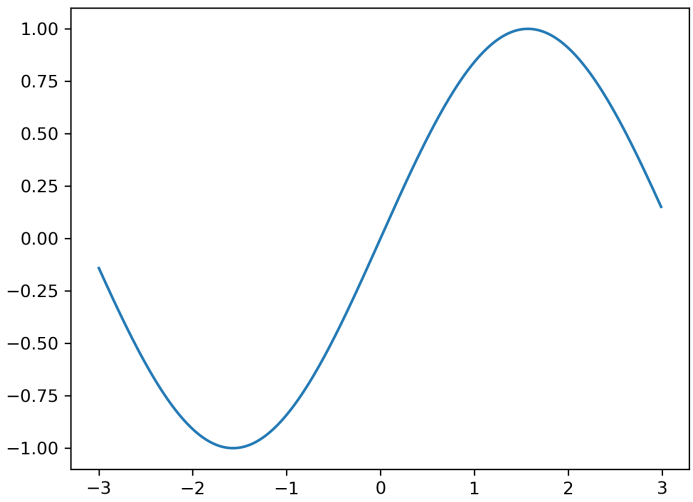

Figure 1: 簡單的 sin 圖形

3.2.2 練習一：繪製折線圖

利用 list 繪製

x1 = [1, 2, 3, 4, 5, 6]

y1 = [1, 2, 3, 4, 5, 6]

執行 plt.plot(y1) 和 plt.plot(x1, y1) 有何差別??

利用 numpy 繪製

x2 = np.arange(0, 3, 0.01)

執行 plt.plot(x2, x2**2) 看看畫出什麼圖形??

3.2.3 圖形美化：plot() function

1. plot() 函式參數可分為兩類：fmt 字串和 kwarg 參數
   - fmt 定義基本格式：顏色、標記、線條樣式

- kwarg 為 Line2D 屬性 (參考資料)：color, linestyle, marker, label, linewidth, ⋯.
- 若 fmt 和 kwarg 設定衝突時，以 kwarg 為主
- plot(): fmt 字串

字元	顏色	字元	標記	字元	標記
‘b’	藍	’.’	點	‘*’	*
‘g’	綠	’o’	圓圈	‘+’	+
‘r’	紅	’v’	三角形 (下)	‘x’	x
‘c’	青	’^’	三角形 (上)	‘d’	鑽石
‘m’	洋紅	‘<’	三角形 (左)	字元	線條
‘y’	黃	‘>’	三角形 (右)	‘-’	實線
‘k’	黑	‘s’	正方形	‘–’ /’:’	虛線
‘w’	白	‘p’	五邊形	‘-.’	-.-.-.-.

2. kwgarg 參數
   - colorg
     - 單字，如 g：color = ‘lime’
     - 字母，如 g：color = ‘k’
     - 色碼，如 g：color = ‘#FF0000’
     - RGB 值 (0~g1 之間)，如：color = (1, 0, 0)
   - label：g 呈現線條標籤，如 label = ‘$y = x^2$’
     - 需搭配 pglt.legend() 函式方能呈現 label
3. 色彩對照

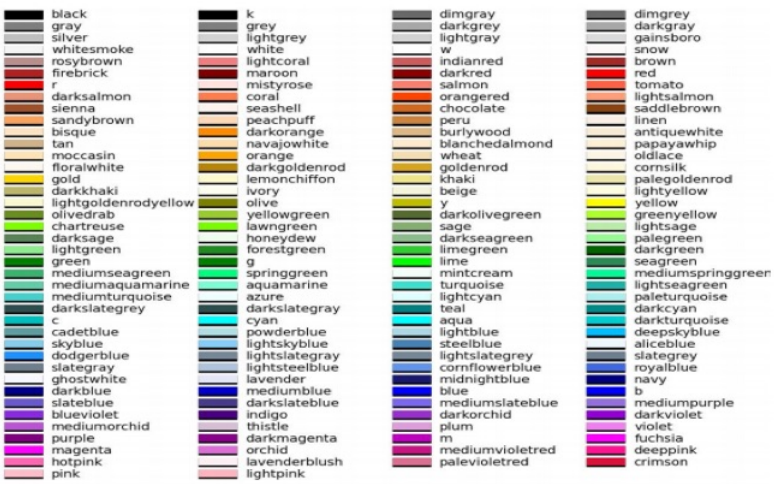

Figure 2: 色表

```
import matplotlib.pyplot as plt
import numpy as np
```

```

x1 = [1, 2, 3, 4, 5, 6]
y1 = [1, 2, 3, 4, 5, 6]
plt.clf()
plt.plot(x1, y1, color='c', linestyle='-', marker='p')
plt.savefig('line1.png', dpi=300)

x2 = np.arange(0, 3, 0.01)
x3 = np.arange(8).reshape(2, 4)

plt.plot(y1, 'c-p', x2, x2**2, x3, x3+3, '-*')
plt.savefig('line2.png', dpi=300)
```

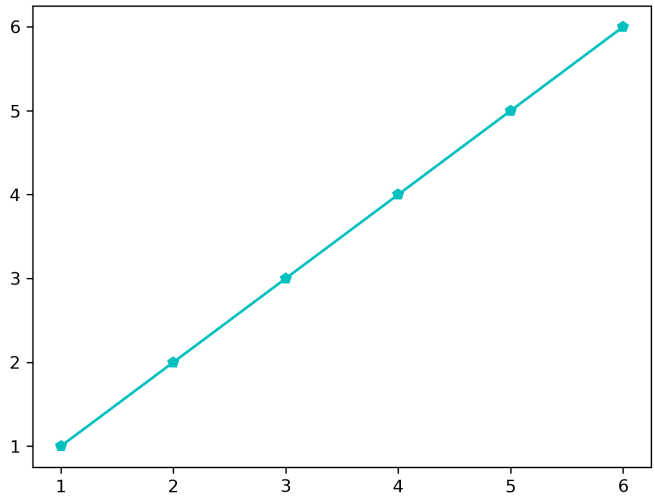

Figure 3: 簡單的折線圖形 #1

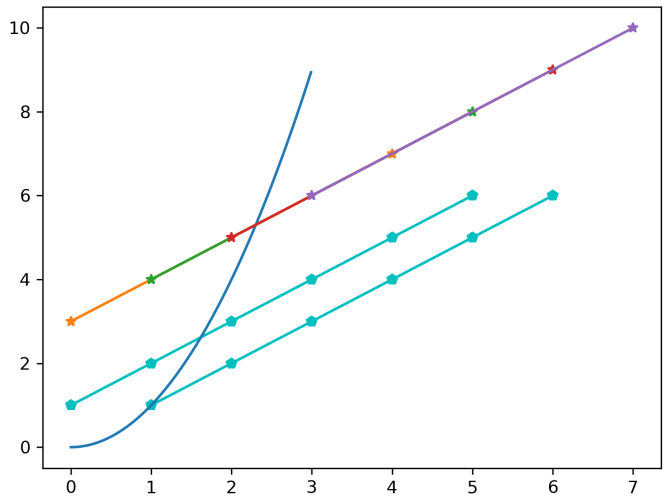

Figure 4: 簡單的折線圖形 2

4. 其他參數

- x / y 座標範圍：plt.xlim(起始值, 終止值) / plt.ylim(起, 止)
- 圖表標題：plt.title(字串)
- x / y 座標標題：plt.xlabel(字串) / plt.ylabel(字串)
- 顯示 kwarg 參數裡的 label：plt.legend()

3.2.4 練習 2：圖表美化

1. 將練習 1 的圖表加上標記、線條標籤，換線條顏色、線條樣式
2. 請將下列資料繪成折線圖 (男女折線不同顏色、樣式，加標記)

初婚年齡	2006	2011	2014	2015	2016
男	30.7	31.8	32.1	32.2	32.4
女	27.8	29.4	29.9	30.0	30.0

- 圖表標題: Age of first marriage
- 座標軸標題: x　Year、y　Age
- 線條標籤: 男　Male、女　Female

3.3 bar chart: plt.bar()

3.3.1 語法

3.3.2 練習 3：

1. 請將練習 2 的男女初婚年齡改成長條圖

3.4 pie chart: plt.pie()

3.4.1 語法：

plt.pie(比例列表 [, 參數 1, 參數 2, ...])

3.4.2 參數：

- colors：各子圖顏色，多以 list 表示
- labels：各子圖標籤，多以 list 表示
- explode：各子圖分離突出比例，0.1 代表分離 10%，多以 list 表示
- autopct：顯示各子圖比例值，格式為%x.y%%
- startangle：繪製起始角度，預設為 0 (與三角函數角度相同)
- 若要以正圓形繪製，請再加上 plt.axis('equal')

3.4.3 範例

```
import matplotlib.pyplot as plt
import numpy as np

parts = [35.35, 23, 26.65, 15]
labels = ['Harrison', 'Vanessa', 'James', 'Ruby']
colors = ['red', 'lightblue', 'purple', 'yellow']
explodes = [0.1, 0, 0, 0.1]
plt.pie(parts, colors = colors, labels = labels, explode = explodes, autopct = '%3.2f%%')
plt.axis('equal')
plt.legend()

#plt.show()
plt.savefig('simplePie.png', dpi=300)
```

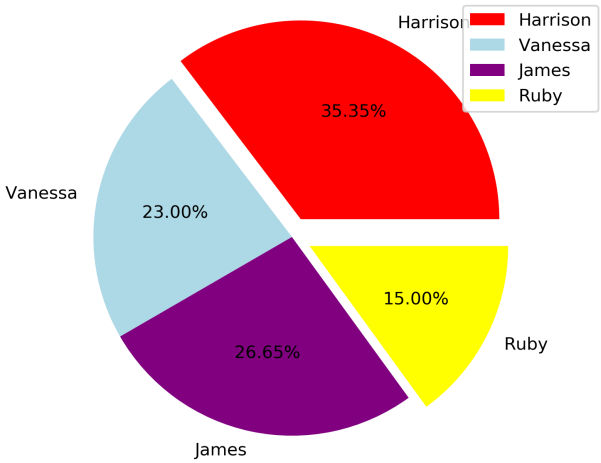

Figure 5: 簡單的 pie chart

3.5 其它圖表

- 直方圖 (Histogram)：用來呈現各資料統計結果 (參考資料)
- 散佈圖 (Scatter)：用來呈現群聚關係 (參考資料)
- 進階學習資源

3.6 文字註解：plt.text()

3.6.1 語法

-
- 參考資料

3.6.2 範例

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5, 6, 7, 8]
y = [1, 4, 9, 16, 25, 36, 49, 64]
plt.plot(x, y, 'r--')
for x, y in zip(x, y):
    plt.text(x-0.2, y+0.6, '(%d, %d)' %(x, y))
#plt.show()
plt.savefig('simpleText.png', dpi = 300)
```

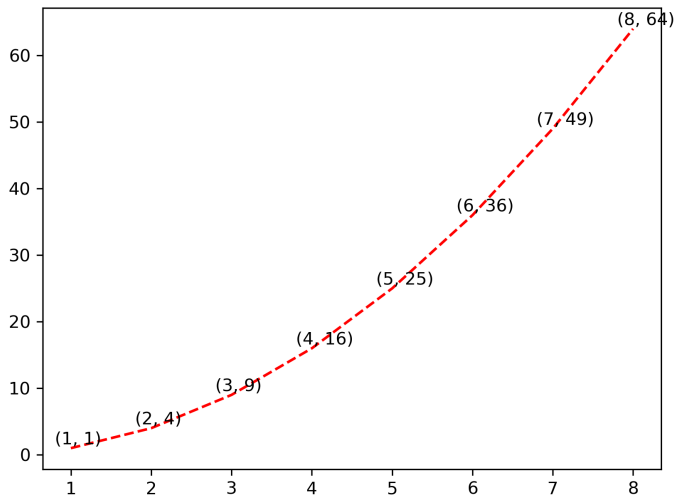

Figure 6: 簡單的文字註解

3.7 子圖表: plt.subplot()

- 範例

```
import numpy as np
import matplotlib.pyplot as plt

from matplotlib.ticker import NullFormatter  # useful for `logit` scale

# Fixing random state for reproducibility
np.random.seed(19680801)

# make up some data in the interval ]0, 1[
y = np.random.normal(loc=0.5, scale=0.4, size=1000)
y = y[(y > 0) & (y < 1)]
y.sort()
x = np.arange(len(y))

# plot with various axes scales
plt.figure()

# linear
plt.subplot(221)
```

```python
plt.plot(x, y)
plt.yscale('linear')
plt.title('linear')
plt.grid(True)


# log
plt.subplot(222)
plt.plot(x, y)
plt.yscale('log')
plt.title('log')
plt.grid(True)


# symmetric log
plt.subplot(223)
plt.plot(x, y - y.mean())
plt.yscale('symlog', linthreshy=0.01)
plt.title('symlog')
plt.grid(True)

# logit
plt.subplot(224)
plt.plot(x, y)
plt.yscale('logit')
plt.title('logit')
plt.grid(True)
# Format the minor tick labels of the y-axis into empty strings with
# `NullFormatter`, to avoid cumbering the axis with too many labels.
plt.gca().yaxis.set_minor_formatter(NullFormatter())
# Adjust the subplot layout, because the logit one may take more space
# than usual, due to y-tick labels like "1 - 10^{-3}"
plt.subplots_adjust(top=0.92, bottom=0.08, left=0.10, right=0.95, hspace=0.25,
                    wspace=0.35)

plt.savefig('simpleSubplot.png', dpi = 300)
```

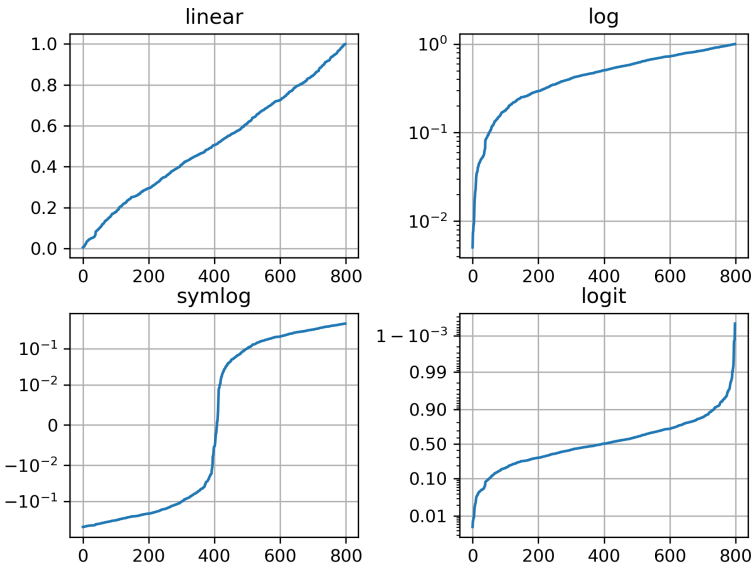

Figure 7: 簡單的子圖表

- 參考資料

4 區域資料檔分析

4.1 Excel/CSV: Pandas 模組

4.1.1 CSV

為純文字檔，以逗號分隔值（Comma-Separated Values，CSV，有時也稱為字元分隔值，因為分隔字元也可以不是逗號）。
純文字意味著該檔案是一個字元序列，不含必須像二進位制數字那樣被解讀的資料。
CSV 檔案由任意數目的記錄組成，記錄間以某種換行符分隔；每條記錄由欄位組成，欄位間的分隔符是其它字元或字串，最常見的是逗號或製表符。通常，所有記錄都有完全相同的欄位序列[7]。

4.1.2 Pandas

Pandas 是 python 的一個數據分析模組，2009 年底開源出來，提供高效能、簡易使用的資料格式 (Data Frame) 讓使用者可以快速操作及分析資料，Pandas 強化了資料處理的方便性也能與處理網頁資料與資料庫資料等，有點類似於 Office 的 Excel 能更加方便的進行運算、分析等[8]。
Pandas 主要特色有[9]：

1. 在異質數據的讀取、轉換和處理上，都讓分析人員更容易處理，例如：從列欄試算表中找到想要的值。

2. Pandas 提供兩種主要的資料結構，Series 與 DataFrame。Series 顧名思義就是用來處理時間序列相關的資料 (如感測器資料等)，主要為建立索引的一維陣列。DataFrame 則是用來處理結構化 (Table like) 的資料，有列索引與欄標籤的二維資料集，例如關聯式資料庫、CSV 等等。

3. 透過載入至 Pandas 的資料結構物件後，可以透過結構化物件所提供的方法，來快速地進行資料的前處理，如資料補值，空值去除或取代等。

4. 更多的輸入來源及輸出整合性，例如：可以從資料庫讀取資料進入 Dataframe，也可將處理完的資料存回資料庫。

4.1.3 Pandas 資料結構

Pandas 是 Python 的資料分析函式庫，從 2009 年底開放原始碼，提供簡易使用的資料結構和分析工具，讓分析人員可以快速方便的處理結構化資料。要使用 Pandas 之前，先瞭解兩

[7] Python 讀寫 csv 檔案的幾種方法及 pandas.read$_{\text{csv 引數全解}}$

[8] Day23- 資料處理模組-Pandas 介紹

[9] PythonPandas 基礎教學

個主要資料結構[10]：

- Series: 是一維標籤陣列（array），能夠保存任何資料類型（整數、字符串、浮點數等）。
- DataFrame: 是一個二維標籤資料結構，可以具有不同類型的行（column），類似 Excel 的資料表，對於有使用過統計軟體的分析人員應該不陌生。

簡單來說，Series 可以想像為一行多列（row）的資料，而 DataFrame 是多行多列的資料，藉由選擇索引（列標籤）和行（行標籤）的參數來操作資料，就像使用統計軟體透過樣本編號或變項名稱來操作資料。

4.2 Pandas functions

4.2.1 資料選取[11]

1. Select as dictinoary (column):
   -
   -

```
import pandas as pd

# 讀取csv
df = pd.read_csv("scores.csv")

# colum
print(df.id)
print(df[['id', 'typing']])
```

2. Select using index (row):
   -

```
import pandas as pd

df = pd.read_csv("scores.csv")

print(df[1:3])
print(df[df.homework<30])
```

3. loc, iloc, between [12]
   - loc: 基於行標籤和列標籤（x_{label}、y_{label}）進行索引，以 column 名做為 index
   - iloc: 基於行索引和列索引（index，columns）都是從 0 開始，以數字做為 index
   - between: 檢查區間值,

   (a) 範例 1

```
import pandas as pd

```

[10] 使用 Python 進行資料整理-初探 Pandas

[11] 實用但常忘記的 Pandas Dataframe 常用指令 1

[12] 使用 Python 進行資料整理-初探 Pandas

```python
df = pd.read_csv("scores.csv")

print(df.loc[df.homework<25, ['id', 'homework']])
print(df.iloc[:5, 2:5])
```

(b) 範例 2

```python
# 載入函式庫
import pandas as pd

print("Pandas version:", pd.__version__)
# 讀取csv
df = pd.read_csv("scores.csv")
# 輸出前3筆
print(df.iloc[:3])
# 輸出前3筆的第2,3欄
print(df.iloc[:2, 1:3])
# 輸出第3筆user的homework成績
print(df.loc[2,'homework'])

print(df['midexam'].between(50, 60))
midFilter = df['midexam'].between(50, 55)
print(df[midFilter])
```

4.2.2 條件式選取資料

1. 語法
   - $\text{src}_{\text{python}}$ [:exports code]{ df[(condition)] }
   - $\text{src}_{\text{python}}$ [:exports code]{ df[(condition 1) & (condition 2)] }
2. 範例

```python
import pandas as pd

df = pd.read_csv("scores.csv")

print(df[(df.midexam >= 100) & (df.finalexam >= 100)])
```

4.3 進階分析

4.3.1 計算: Aggregation, Groupby

1. create new column based on other column
2. simple aggregation
3. groupby with aggregation function
4. 範例

```python
# 載入函式庫
import pandas as pd

# 讀取csv
df = pd.read_csv("scores.csv")

# 計算總分'
df['Final'] = df.classparti*.1 + df.typing*.1 + df.homework*.2 + df.midexam
    *.3 + df.finalexam*.3

```

```
# create new column according to typing speed
df.loc[df.typing < 30, 'TypingSpeed' ] = 'LOW'
df.loc[df.typing.between(30, 60), 'TypingSpeed' ] = 'MID'
df.loc[df.typing > 60, 'TypingSpeed' ] = 'HIGH'

print(df)

# 依打字速度分組
print(df.groupby('TypingSpeed')['Final'].mean())

# PASS/FAIL
df.loc[df.Final < 60, 'PASS'] = False
df.loc[df.Final >= 60, 'PASS'] = True

print(df.groupby('PASS')['typing'].mean())
```

4.3.2 資料檔下載: scores.csv

4.4 資料視覺化

4.4.1 Bar chart

```
# 載入函式庫
import pandas as pd
import matplotlib.pyplot as plt
# 讀取csv
df = pd.read_csv("scores.csv")

# 計算總分'
df['Final'] = df.classparti*.1 + df.typing*.1 + df.homework*.2 + df.midexam*.3 +
    df.finalexam*.3

# create new column according to typing speed
df.loc[df.typing < 30, 'TypingSpeed' ] = 'Level-1'
df.loc[df.typing.between(30, 60), 'TypingSpeed' ] = 'Level-2'
df.loc[df.typiohong > 60, 'TypingSpeed' ] = 'Level-3'

fig = df.groupby('TypingSpeed')['homework','classparti'].mean().plot(kind='bar',
    title="XXX", rot=0, legend=True)
fig.set_xlabel("Typing Speed")
fig.set_ylabel("score")
savefig = fig.get_figure()
savefig.savefig('pandasPlot1.png', bbox_inches='tight')
```

4.4.2 Pie chart

```
# 載入函式庫
import pandas as pd
import matplotlib.pyplot as plt
# 讀取csv
df = pd.read_csv("scores.csv")

# 計算總分'
df['Final'] = df.classparti*.1 + df.typing*.1 + df.homework*.2 + df.midexam*.3 +
    df.finalexam*.3

# create new column according to typing speed
```

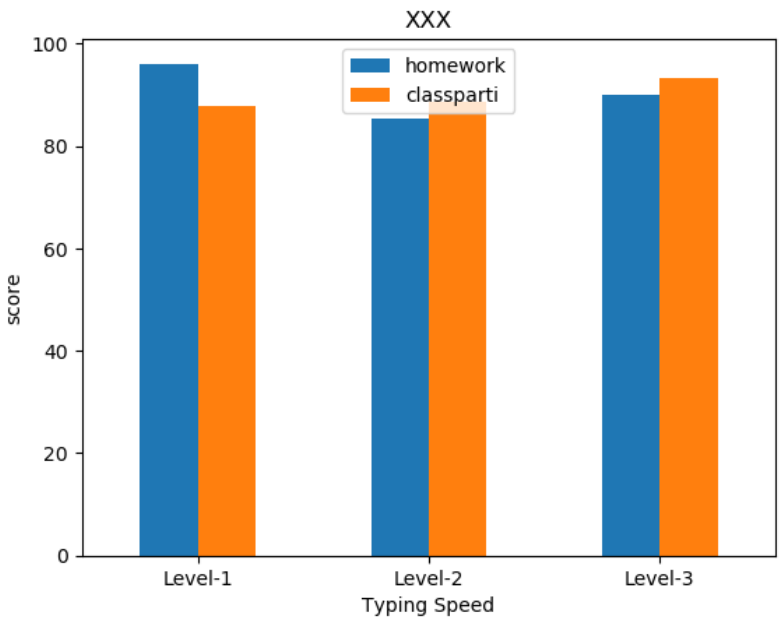

Figure 8: Pandas plot bar chart

```
df.loc[df.typing < 60, 'TypingSpeed' ] = 'Level-1'
df.loc[df.typing.between(60, 80), 'TypingSpeed' ] = 'Level-2'
df.loc[df.typing > 80, 'TypingSpeed' ] = 'Level-3'
print(df.groupby('TypingSpeed').count())

fig = df.groupby('TypingSpeed')['TypingSpeed'].count().plot(kind='pie', title="
    XXX", rot=0, legend=True)
savefig = fig.get_figure()
savefig.savefig('pandasPlot2.png', bbox_inches='tight')
```

4.4.3 Scatter chart

```
# 載入函式庫
import pandas as pd
import matplotlib.pyplot as plt

# 讀取csv
df = pd.read_csv("scores.csv")

fig = df[['typing','homework']].plot(kind='scatter', x=0, y=1)
df[['typing','homework']].plot.line(x=0, y=1)
#fig = df[['typing','finalexam']].plot(kind='scatter', x=0, y=1)

savefig = fig.get_figure()
savefig.savefig('PandasPlot3.png')
```

4.4.4 Line chart

```
# 載入函式庫
import pandas as pd
import matplotlib.pyplot as plt

# 讀取csv
df = pd.read_csv("scores.csv")
```

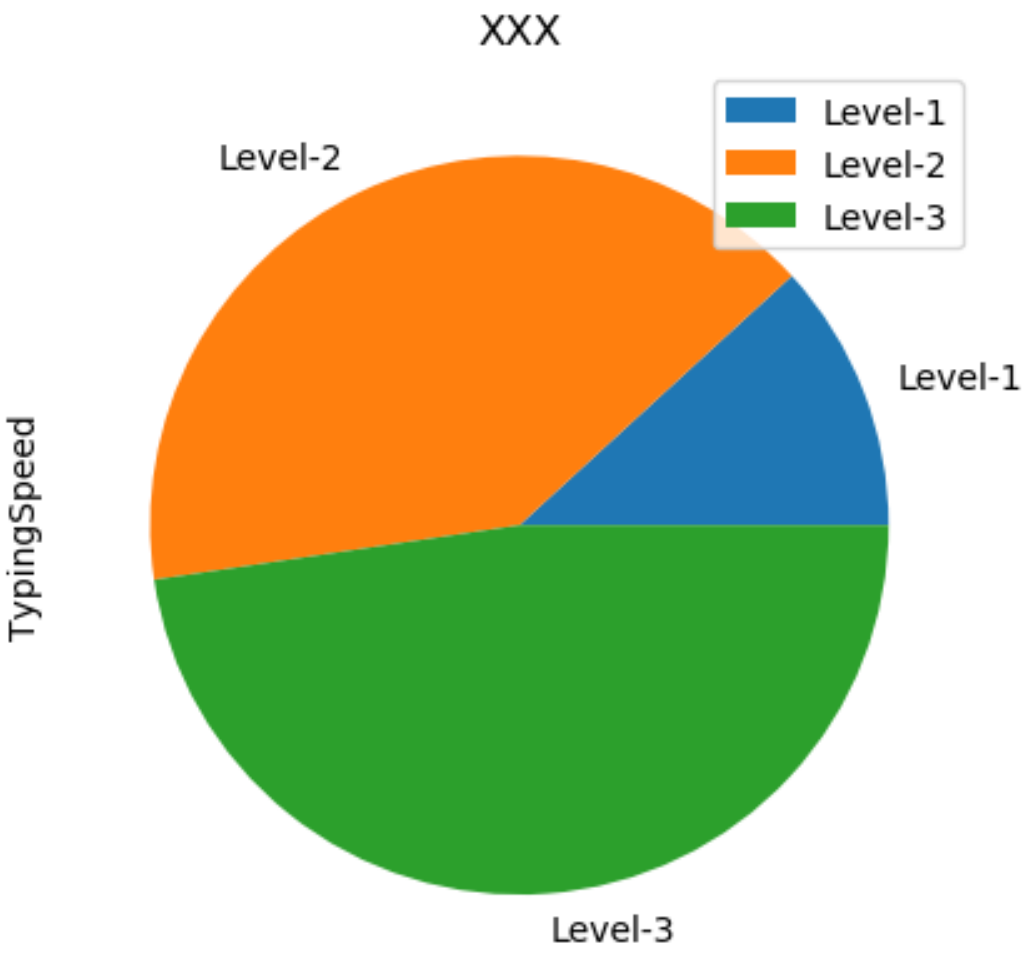

Figure 9: Pandas plot pie chart

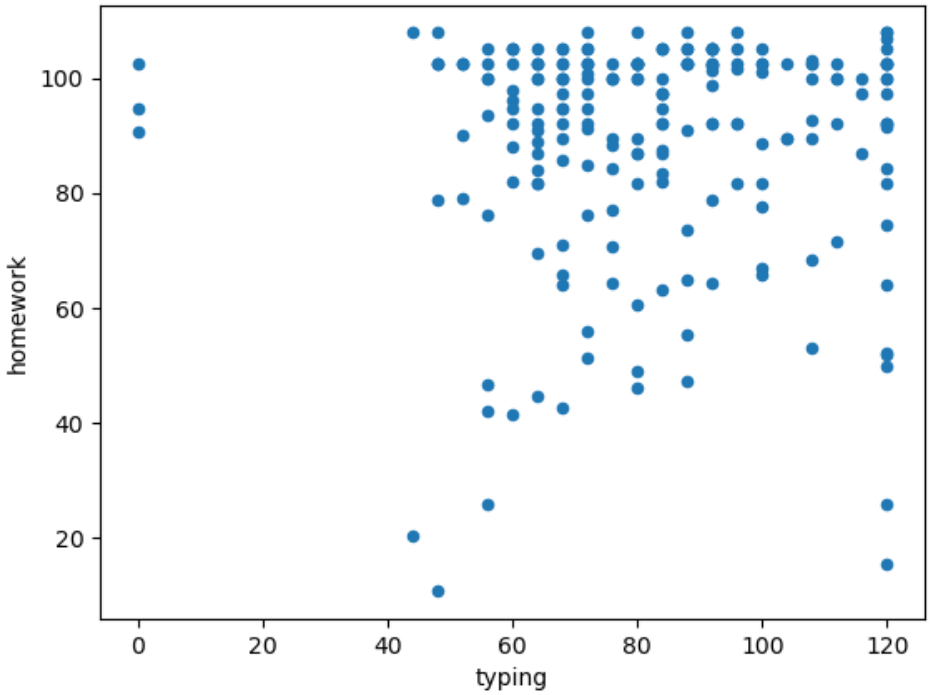

Figure 10: Pandas plot scatter chart

```
fig = df[['midexam','finalexam']].plot(kind='scatter', x=0, y=1)

savefig = fig.get_figure()
savefig.savefig('PandasPlot4.png')
```

5 網路爬蟲

5.1 parse JSON

5.1.1 What is JSON?[13]

JSON(JavaScript Object Notation，JavaScript 物件表示法) 是個以純文字為基底去儲存和傳送簡單結構資料，你可以透過特定的格式去儲存任何資料 (字串, 數字, 陣列, 物件)，也可以透過物件或陣列來傳送較複雜的資料。一旦建立了您的 JSON 資料，就可以非常簡單的跟其他程式溝通或交換資料，因為 JSON 就只是純文字個格式。

5.1.2 JSON 的優點

- 相容性高
- 格式容易瞭解，閱讀及修改方便

[13]你不可不知的 JSON 基本介紹

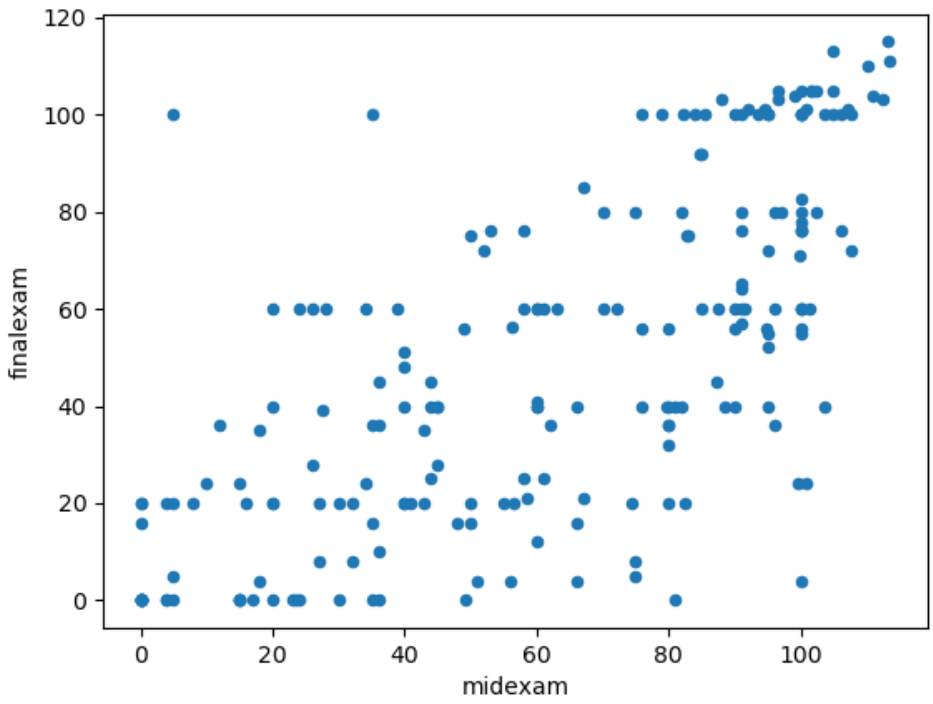

Figure 11: Pandas plot scatter chart

- 支援許多資料格式 (number,string,booleans,nulls,array,associative array)
- 許多程式都支援函式庫讀取或修改 JSON 資料

5.1.3 JSON 結構

- 物件: {}
- 陣列: []

5.1.4 實作 1: 國際主要國家貨幣每月匯率概況

- 下載 JSON

```
import requests

json_url = 'https://apiservice.mol.gov.tw/OdService/download/A17030000J
    -000049-8nD'
response = requests.get(json_url)

jsonRes = response.json()
print(jsonRes)
print(type(jsonRes))
for item in jsonRes:
    print(item['月別'], ": ", item['新台幣'], " / ", item['歐元'])
```

- 資料視覺化

```
import requests
import pandas as pd

json_url = 'https://apiservice.mol.gov.tw/OdService/download/A17030000J-000049-8nD'
response = requests.get(json_url)
jsonRes = response.json()

dates = [i['月別'] for i in jsonRes]
NT = [float(i['新台幣']) for i in jsonRes]
JPY = [float(i['日圓']) for i in jsonRes]
EUR = [float(i['歐元']) for i in jsonRes]
df = pd.DataFrame({'月份':dates, '新台幣':NT, '日圓':JPY, '歐元':EUR})
df['月份'] = [pd.to_datetime(i, format='%y%m', errors='ignore') for i in df['月份']]
print(df)

import matplotlib.pyplot as plt

plt.clf()
plt.plot(df['月份'], NT)
#plt.plot(dates, JPY)
#plt.plot(dates, EUR)
plt.savefig('jsonLine.png', dpi=300)
```

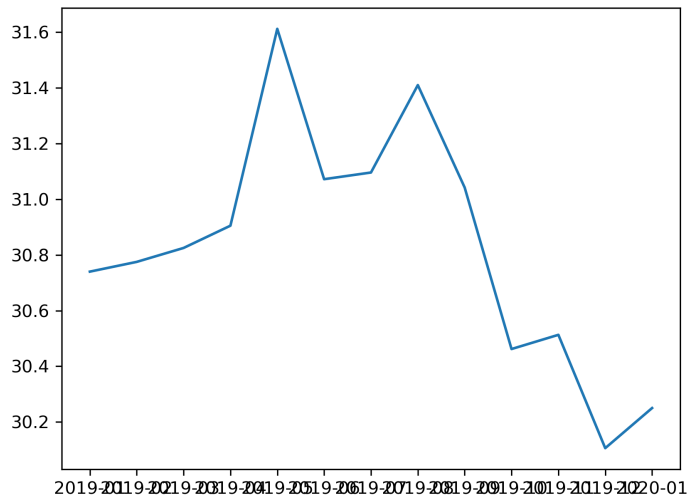

Figure 12: 滙率

5.1.5 實作 2: 澎湖生活博物館每月參觀人次統計資料

1. 中文亂碼處理
   - Python-MacOS 上 matplotlib 顯示中文字體
   - 關於 python：Matplotlib 使刻度標籤字體更小
2. DEMO

```python
#coding:utf-8
import requests
import pandas as pd
from datetime import datetime

json_url = 'http://opendataap2.penghu.gov.tw/resource/files/2020-01-12/eaa641fc3af66277e60b13201ca11232.json'

response = requests.get(json_url)
jsonRes = response.json()

year = [i['年度'] for i in jsonRes]
month = [i['月份'] for i in jsonRes]
visitor = [int(i['人數']) for i in jsonRes]

df = pd.DataFrame({'year':year, 'month':month, 'visitor':visitor})
df['date'] = [str(int(i)+1911)+'-'+j for i, j in zip(df['year'], df['month'])]

df['time'] = [pd.to_datetime(i, format='%y%m', errors='ignore') for i in df['date']]

import matplotlib.pyplot as plt
plt.rcParams['font.family'] = 'cwTeXFangSong'
plt.clf()
plt.bar(df['date'], df['visitor'])
plt.xticks(rotation=60, fontsize=6)
plt.ylabel('人數')
plt.xlabel('日期')
plt.title('澎湖生活博物館每月參觀人次')
plt.savefig('jsonBar.png', dpi=300, bbox_inches='tight')
```

5.1.6 實作 3: 下載 JSON

```python
import requests
import json
json_url = 'http://www.kh.edu.tw/json/bulletin/employ/datagrid?page=1&rows=20'
response = requests.get(json_url)

print(response)
# 方法1
jsonRes1 = response.json()
print(type(jsonRes1))
#
jsonRes2 = json.loads(response.text)
print(type(jsonRes2))

print(jsonRes2)
```

5.1.7 實作: 解析 JSON

```python
import requests
import json

response = requests.get('http://www.kh.edu.tw/json/bulletin/employ/datagrid?page=1&rows=20')
jsonRes = response.json()
```

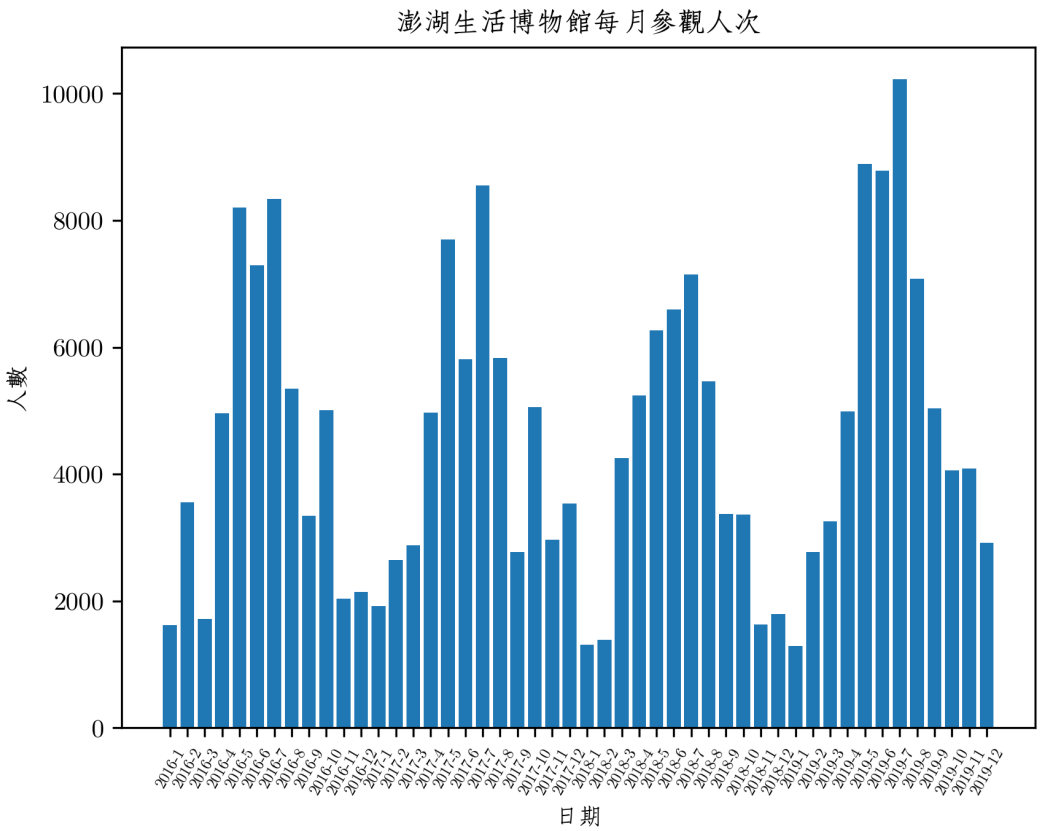

Figure 13: 參觀人數

```
#print(jsonRes['rows'])
for item in jsonRes['rows']:
    print(item['author'], ":", item['title'],"  / ", item['pubDate'])
sortJR = jsonRes['rows']
print(type(sortJR))

sortJR.sort(key=lambda x: x2['author'], reverse=False)
for item in sortJR:
    print(item['author'], ":", item['title'],"  / ", item['pubDate'])
```

5.1.8 練習

以程式抓取高雄市政府資料開放平台的線上 JSON 資料，parse 並輸出所需結果

5.2 parse HTML

- what is HTML
- 精神：將程式模仿成一般 user

5.2.1 所需套件

- BeautifulSoup

pip install beautifulsoup4

5.3 範例: 以程式抓取 PTT 電影板 header

5.3.1 原則

1. 要抓網路資料，就要先仔細觀察網頁內容

2. 要儘量欺騙網站自己是 browser

5.3.2 實作: 取得 HTML 資料

1. 第一版

```
# 抓取PTT電影版header
import urllib.request as req
url = "https://www.ptt.cc/bbs/Stock/index.html"

with req.urlopen(url) as response:
    data = response.read().decode("utf-8")
print(data)
```

被拒絕原因：這隻程式的行為不像一般使用者，被網站伺服器拒絕。403 Forbidden 是 HTTP 協議中的一個 HTTP 狀態碼（Status Code）。可以簡單的理解為沒有權限訪問此站，服務器收到請求但拒絕提供服務[14]。

2. 第二版: 加入 header

```
import urllib.request as req
url = "https://www.ptt.cc/bbs/Stock/index.html"
# 幫request加上一個header
request = req.Request(url, headers = {
    "User-Agent":"Mozilla/5.0 (Macintosh; Intel Mac OS X 10.14; rv:72.0)
        Gecko/20100101 Firefox/72.0"
})

with req.urlopen(request) as response:
    data = response.read().decode("utf-8")
print('所取得的資料型態: ',type(data))
print(data[700:1000])
```

5.3.3 實作：解析 HTML 資料內容

1. 安裝解析 HTML 所需套件

```
pip install beautifulsoup4
```

2. 第三版: 解析 HTML

```
import urllib.request as req
url = "https://www.ptt.cc/bbs/Stock/index.html"
# 幫request加上一個header
request = req.Request(url, headers = {
```

[14]維基百科: HTTP 403

```
    "User-Agent":"Mozilla/5.0 (Macintosh; Intel Mac OS X 10.14; rv:72.0) Gecko/20100101 Firefox/72.0"
})

with req.urlopen(request) as response:
    data = response.read().decode("utf-8")

import bs4
root = bs4.BeautifulSoup(data,"html.parser")
print(root.title)
titles = root.find("div", class_="title")
print(titles)
print(titles.a.string)
titles = root.find_all("div", class_="title")
print(titles)
for title in titles:
    if title.a != None:
        print(title.a.string)
```

5.4 參考資料

- https://www.youtube.com/watch?v=9Z9xKWfNo7k

6 Python 學習資源

- Python 3 教程
- Python 教程

7 0802 CNN

https://reurl.cc/Y2gKo: 各種常用的 library, function

8 如何把 python 程式 publish 成網站應用程式